

扫码获取
答案解析

提高组 CSP-S 2025 初赛模拟卷 5

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. C++ 语言中析构函数的主要作用是（ ）。
A. 为对象分配内存空间
B. 对对象的静态数据成员进行初始化
C. 重载对象的运算符
D. 在对象生命周期结束时释放资源，执行清理任务
2. 以下关于 NOI Linux 的文件操作需要十分谨慎的是（ ）。
A. `rm -r` B. `cp -a` C. `mkdir` D. `diff`
3. 以下哪一项并非 STL 中集合容器所具备的操作函数？（ ）
A. `insert` B. `swap` C. `front` D. `lower_bound`
4. 设 $A=B=\text{true}$, $C=D=\text{false}$, 以下逻辑运算表达式的值为假的是（ ）。
A. $(\neg A \wedge B) \vee (C \wedge D \vee A)$ B. $\neg(((A \wedge B) \vee C) \wedge D)$
C. $A \wedge (B \vee C \vee D) \vee D$ D. $(A \wedge (D \vee C)) \wedge B$
5. 阅读以下用动态规划解决 0-1 背包问题的函数，假设背包的容量是 10kg, 4 个物品的 a （重量）分别为 1,3,4,6（单位为 kg），每个物品对应的 b （价值）分别为 20,30,50,60，则函数的输出为（ ）。

```
int knapsack(int W, const vector<int>& a, const vector<int>& b, int n) {  
    vector<vector<int>>> dp(n+1, vector<int>(W+1, 0));  
    for (int i=1; i<=n; i++) {  
        for (int w=0; w<=W; w++) {  
            if (a[i-1] <= w) {  
                dp[i][w] = max(dp[i-1][w], dp[i-1][w-a[i-1]]+b[i-1]);  
            }  
            else  
            {  
                dp[i][w] = dp[i-1][w];  
            }  
        }  
    }  
    return dp[n][W];  
}
```

```

    }
    }
    return dp[n][W];
}

```

A. 90 B. 100 C. 110 D. 140

6. 给定一棵二叉树, 其前序遍历结果为 1245367, 中序遍历结果为 4521367, 则这棵树的后序遍历结果为 ()。

A. 5427631 B. 5472631 C. 4527631 D. 4257631

7. 一棵有 n 个结点的二叉树, 执行释放全部结点操作的时间复杂度是 ()。

A. $O(\log n)$ B. $O(n^2)$ C. $O(n)$ D. $O(n \log n)$

8. 下列选项中, () 不是动态规划算法中常见的优化手段。

A. 滚动数组 B. 单调队列 C. 自动机 D. 状态压缩

9. 下面哪个说法是错误的? ()

A. 向量又称矢量
B. 多重集合不是集合
C. 高斯消元法可以求逆矩阵和行列式
D. 矩阵只能做加减运算, 不能做乘法运算

10. 设正整数 $1 \leq n \leq 2025$, 则使 $n^5 - 5n^3 + 4n + 15$ 是完全平方数的可能的 n 的个数为 () 个。

A. 3 B. 0 C. 1 D. 2

11. 以下哪种数据结构是线性数据结构? ()

A. trie B. Treap C. deque D. Splay

12. 冒泡排序、选择排序、插入排序、归并排序、快速排序、堆排序和桶排序算法中有 () 个是稳定的。

A. 3 B. 4 C. 5 D. 2

13. 对于一个正整数 n , 如果能找到正整数 a 和 b 使得 $n=a+b+a*b$, 则称 n 为奇特数, 例如 $3=1+1+1*1$ 就是一个奇特数, 从 1 到 100 中有 () 个奇特数。
A. 74 B. 72 C. 73 D. 75
14. 现有 7 把钥匙和 7 把锁, 用这些钥匙随机开锁, 则 A1,A2,A3 这三把钥匙全部不能打开对应锁的概率是 ()。
A. 3/7 B. 67/105 C. 12/35 D. 9/49
15. 以下哪种方法不是解决哈希冲突的方法? ()
A. 开哈希法 B. 线性探查法 C. 二次探查法 D. 复杂哈希函数

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include <bits/stdc++.h>
02
03 #define N 1005
04 using namespace std;
05
06 int n,m,t,fi[N],cnt,f[N][N],mh[N],ans;
07 bool vis[N];
08 char c[N];
09
10 struct p{
11     int ne,to;
12 } l[N*N];
13
14 struct data {
15     int x,y;
16 } a[N];
17
18 inline void add(int x,int y) {
19     l[++cnt].ne=fi[x];
20     l[cnt].to=y;
21     fi[x]=cnt;
22 }
23
```

```

24 inline int read() {
25     int res=0,f=1;char ch=getchar();
26     while(ch<'0' || ch>'9')f=ch=='-'?-1:f,ch=getchar();
27     while(ch>='0' && ch<='9')res=res*10+ch-'0',ch=getchar();
28     return res*f;
29 }
30
31 inline bool dfs(int u) {
32     for(int i=fi[u];i;i=l[i].ne) {
33         int v=l[i].to;
34         if(vis[v])continue;
35         vis[v]=1;
36         if(!mh[v] || dfs(mh[v])) {
37             mh[v]=u;
38             return 1;
39         }
40     }
41     return 0;
42 }
43
44 int main() {
45     n=read(),m=read();
46     for(int i=1;i<=n;++i) {
47         scanf("%s",c+1);
48         for(int j=1;j<=m;++j)
49             if(c[j]=='1')a[++t].x=i,a[t].y=j;
50     }
51     for(int i=1;i<=t;++i)
52         for(int j=i+1;j<=t;++j)
53             if((a[i].y+1==a[j].y && a[i].x==a[j].x) || (a[i].y==a[j].y
54                 && a[i].x+1==a[j].x)) f[i][j]=1;
55     for(int i=1;i<=t;++i)
56         for(int j=1;j<=t;++j)
57             f[i][j]=f[i][k] & f[k][j];
58     for(int i=1;i<=t;++i)
59         for(int j=i+1;j<=t;++j)
60             if(f[i][j])add(i,j+t);
61     for(int i=1;i<=t;++i) {
62         memset(vis,0,sizeof(vis));
63         if(dfs(i))++ans;

```

```

64     }
65     printf("%d\n", t-ans);
66     return 0;
67 }

```

假设 $1 \leq N, M \leq 20$, 输入 1 的个数不会超过 200 个, 回答下列问题。

■ 判断题

16. 这段代码的时间复杂度为 $O(N^2)$ 。 ()
17. 这段代码的空间复杂度为 $O(N^2)$ 。 ()
18. t 的最大值为 $n*m$ 。 ()

■ 选择题

19. 若读入

```

5 5
00100
11111
00100
11111
00100

```

则程序运行跳出第 51 行的循环后, 有多少个 $f[i][j]$ 的初始值为 1? ()

- A. 10 B. 11 C. 12 D. 13

20. 若读入

```

5 5
10101
10101
01110
11111
00100

```

则程序运行跳出第 51 行的循环后, 有多少个 $f[i][j]$ 的初始值为 1? ()

- A. 13 B. 14 C. 15 D. 16

(2)

```

01 #include <cstdio>
02 #include <iostream>
03 #include <algorithm>

```

```

04 #include <cstring>
05 using namespace std;
06 const int Maxn=3e5+10;
07 char s[Maxn];
08 int a[Maxn],b[Maxn];
09 int f[Maxn];
10 int n;
11 inline bool cmp(int x,int y)
12 {
13     return x>y;
14 }
15 int main()
16 {
17     scanf("%s",s+1);
18     n=strlen(s+1);
19     for(int i=1;i<=n;++i)
20         a[i]=s[i]-'a';
21     scanf("%s",s+1);
22     for(int i=1;i<=n;++i)
23         b[i]=s[i]-'a';
24     sort(a+1,a+1+n);
25     sort(b+1,b+1+n,cmp);
26     int x=0,y=0,pos=n;
27     for(int i=1;i<=n;++i)
28     {
29         if(a[x+1]>=b[y+1]) {pos=i-1;break;}
30         if(i & 1) f[i]=a[++x];
31         else f[i]=b[++y];
32     }
33     x=y=(n>>1)+1;
34     if((n & 1)) ++x;
35     for(int i=n;i>pos;--i)
36     {
37         if((pos+(n-i+1)) & 1) f[i]=a[--x];
38         else f[i]=b[--y];
39     }
40     for(int i=1;i<=n;++i)
41         putchar(f[i]+'a');
42     putchar('\n');
43     return 0;
44 }

```

假设读入的两个字符串长度 n 满足 $1 \leq n \leq 3e5$, 其他输入均合法, 回答下列问题。

■ 判断题

21. 第 25 行排序的 `cmp` 函数使得 `b` 数组变为从大到小排列, 为达到同样的效果也可以使用 `less<int>`。 ()
22. `std::sort()` 的时间复杂度是随机的, 从 $O(N)$ 到 $O(N^2)$ 都有可能。 ()
23. (2 分) 假设这里用的 `sort` 采用了快速排序, 那么快速排序是稳定的, 即对于 $i < j$, 当 $a_i = a_j$ 时, 排序后 a_i 一定在 a_j 前。 ()
24. (2 分) 若将第 33 行代码 `x=y=(n>>1)+1`; 改为 `x=y=n>>1+1`;, 程序在所有相同输入下输出不变。 ()

■ 选择题

25. 若程序输入

tinkoff

zscoder

则输出为 ()。

- A. fzfsirk B. fzfsidk C. zfsfrik D. Zfsfdik

26. 下列说法中正确的是 ()。

- A. `strlen()` 和 `vector` 的 `size()` 函数的时间复杂度相同, 都为 $O(N)$ 。
- B. 当输入两个相同的串时, 该程序的输出也是这个串。
- C. 当输入两个不同的串 a, b 时, 该程序也可能输出与 a 或 b 相同的串。
- D. 若将这段代码中的 `--y` 全部改成 `y--`, 程序依然可以输出相同的结果。

27. 假设当 $n=100$ 时, 第 37 行中的 `f[i]=a[--x]`; 语句最多被执行多少次? ()

- A. 8 B. 49 C. 50 D. 100

(3)

```
01 #include <bits/stdc++.h>
```

```
02 using namespace std;
```

```
03 typedef long long LL;
```

```
04 const int N = 1e6 + 5, INF = 0x3f3f3f3f;
```

```
05 const LL mod = 1e9 + 7;
```

```
06 int n, l, a, b;
```

```
07 bool flag[N];
```

```
08 int c[N];
09 int dp[N];
10 struct segtree {
11     int l, r, val;
12     #define l(x) tr[x].l
13     #define r(x) tr[x].r
14     #define val(x) tr[x].val
15 } tr[N << 1];
16 void pushup(int x) {
17     val(x) = min(val(x << 1), val(x << 1 | 1));
18 }
19 void build(int l, int r, int x) {
20     l(x) = l, r(x) = r, val(x) = INF;
21     if(l == r) return;
22     int mid = l + r >> 1;
23     build(l, mid, x << 1), build(mid + 1, r, x << 1 | 1);
24 }
25 void update(int pos, int v, int x) {
26     if(l(x) == r(x)) {
27         val(x) = v;
28         return;
29     }
30     int mid = l(x) + r(x) >> 1;
31     if(mid >= pos) update(pos, v, x << 1);
32     else update(pos, v, x << 1 | 1);
33     pushup(x);
34 }
35 int query(int l, int r, int x) {
36     if(l <= l(x) && r(x) <= r) return val(x);
37     int mid = l(x) + r(x) >> 1, res = INF;
38     if(l <= mid) res = query(l, r, x << 1);
39     if(r > mid) res = min(res, query(l, r, x << 1 | 1));
40     return res;
41 }
42 int main()
43 {
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr);
46     cin >> n >> l >> a >> b;
47     for(int i = 1; i <= n; i++) {
48         int x, y;
```

```

49     cin >> x >> y;
50     c[x + 1] ++, c[y] --;
51 }
52 for(int i = 1; i <= l; i++) {
53     c[i] += c[i - 1];
54     if(c[i] > 0) flag[i] = true;
55 }
56 build(0, l, 1);
57 update(0, 0, 1);
58 for(int i = a * 2; i <= l; i += 2) {
59     if(flag[i]) continue;
60     int ql = max(0, i - 2 * b), qr = i - 2 * a;
61     dp[i] = query(ql, qr, 1) + 1;
62     update(i, dp[i], 1);
63 }
64 if(dp[l] >= INF) dp[l] = -1;
65 cout << dp[l] << '\n';
66 return 0;
67 }

```

假设对于输入满足 $1 \leq L \leq 1e6$, $1 \leq A \leq B \leq 1e3$, $1 \leq N \leq 1e3$, $1 \leq S_i < E_i \leq L$; L 一定是一个偶数。

■ 判断题

28. 本段代码第 15 行中的 $N \ll 1$ 应该改成 $N \ll 2$, 否则会发生数组下标越界。 ()

■ 选择题

29. dp 数组的转移方程是 ()。

A. $dp[i] = \min(dp[j]), i - b \leq j \leq i - a$

B. $dp[i] = \min(dp[j]), i - b * 2 \leq j \leq i - a * 2$

C. $dp[i] = \min(dp[j]) + 1, i - b \leq j \leq i - a$

D. $dp[i] = \min(dp[j]) + 1, i - b * 2 \leq j \leq i - a * 2$

30. 若输入

```

2 8
1 2
6 7

```

3 6

则程序输出 ()。

- A. -1 B. 1 C. 2 D. 3

31. 若输入

4 8

1 2

6 7

3 6

2 4

1 8

则程序输出 ()。

- A. -1 B. 1 C. 2 D. 3

32. 以下说法中正确的是 ()。

- A. 第 50 行代码中的 flag 数组标记了 $[x+1, y-1]$ 这个区间。
 B. 第 57 行代码 `update(0, 0, 1);` 可删去, 因为这是对第 0 个元素更新值, 完全没有必要。
 C. 第 58 行代码 `for(int i = a * 2; i <= l; i += 2)` 可以改成 `for(int i = a * 2; i <= l; i += 1)`。
 D. 第 61 行代码中的 `query(ql, qr, 1)` 也可以写成 `query(ql, qr, 0)`。

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) 题目描述:

给你一个 $n \times n$ 的矩阵, 你可以将 $k \times k$ 的连续子矩阵全部加一或者减一, 初始时, 有 m 个位置不是 0, 其他全部是 0, 请问你至少需要多少次操作才能将矩阵中的所有数都变为 0? 如果无法实现, 请输出 -1。

显然, 利用差分性质能够实现快速修改, 具体做法是按照顺序依次修改 $(1, 1)$, $(1, 2), \dots, (1, n), \dots, (n, n)$ 。

```
01 #include <bits/stdc++.h>
02 #define ll long long
03 using namespace std;
04 ll d[5010][5010], f[5010][5010];
05 int n, m, k;
```

```

06 void cf(int x,int y,int c) {
07     int xx=x+k,yy=y+k;
08     if(①) {
09         cout<<"-1";
10         exit(0);
11     }
12     d[x][y]-=c;
13     d[xx][y]+=c;
14     d[x][yy]+=c;
15     ②;
16 }
17 int main() {
18     ll ans=0;
19     scanf("%d%d%d",&n,&m,&k);
20     while(m--) {
21         int x,y;
22         ll z;
23         scanf("%d%d%lld",&x,&y,&z);
24         f[x][y]=z;
25     }
26     for(int i=1;i<=n;i++)
27         for(int j=1;j<=n;j++) {
28             d[i][j]+= ③;
29             ll k=④;
30             if(k) {
31                 ans+=⑤;
32                 cf(i,j,k);
33             }
34         }
35     printf("%lld",ans);
36     return 0;
37 }

```

33. ①处应填 ()。

A. $xx \geq n+1 || yy \geq n+1$

B. $xx > n+1 \& \& yy > n+1$

C. $xx < 1 || yy < 1$

D. $xx < 1 \& \& yy < 1$

34. ②处应填 ()。

A. $d[yy][xx] -= c$

B. $d[yy][xx] += c$

C. $d[xx][yy] -= c$

D. $d[xx][yy] += c$

35. ③处应填 ()。

A. $-d[i-1][j-1]$

B. $d[i][j-1]-d[i-1][j-1]$

C. $d[i-1][j]-d[i-1][j-1]$

D. $d[i-1][j]+d[i][j-1]-d[i-1][j-1]$

36. ④处应填 ()。

A. $d[i][j]$

B. $f[i][j]$

C. $d[i][j]+f[i][j]$

D. $d[i][j]-f[i][j]$

37. ⑤处应填 ()。

A. 1

B. $\text{abs}(k)$

C. k

D. $-k$

(2) 题目描述:

给定一个序列, 选择一个前缀和一个与之不相交的后缀, 使得所有被选择的数的异或和最大。

先计算并且插入所有后缀异或和 b_k , 从前往后记录当前前缀和 cur , 删除后缀异或和并在 Trie 树里面寻找与 cur 对应的异或最大值, 更新答案即可。

```
01 #include <bits/stdc++.h>
02 #define ll long long
03 #define reg register
04 #define F(i,a,b) for(reg int i=(a);i<=(b);i++)
05 inline ll read();
06 using namespace std;
07 const int N=1e5+10;
08 int n,trie[N*64][2],cnt=1,rec[N*64];
09 ll bk[N],a[N],ans;
10 inline void insert(ll x) {
11     int p=1;
12     for(reg int i=62; i>=0; i--) {
13         bool k=(x&(1<math>①</math>));
14         if(!trie[p][k])trie[p][k]=++cnt;
15         p=trie[p][k];
16         ②;
17     }
18 }
19 inline void del(ll x) {
20     int p=1;
21     for(reg int i=62; i>=0; i--) {
```